

L4a: Graph and Tree Representations

CHEME 5800 Cornell University

Objectives for Today

Today we describe graph structure, calculate graph measures, and choose a storage form that matches the operation an algorithm performs.

Today's concepts

- **Graph language and measures**: define walks, paths, cycles, connectivity, degree, and density.
- **Storage representations**: compare edge lists, adjacency matrices, and adjacency lists by storage and access cost.
- **Graph families**: recognize complete graphs, bipartite graphs, and trees from their defining properties.

Lecture: [Graph and Tree Representations](#)

Lecture and Connected Exercises

Lecture: [Graph and Tree Representations](#)

Lab L4b: [Breadth-First and Depth-First Search](#)

Use an unweighted adjacency list to visit a directed graph with a queue or a stack.

Lecture L4c and lab L4d: [Shortest-Path Algorithms](#)

Retain edge weights and find minimum-cost routes through a process network.

Simple Graphs

A simple graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of a set of vertices $\mathcal{V}$ and a set of edges $\mathcal{E}$.

- In an **undirected graph**, an edge $\{u, v\}$ is an unordered pair of distinct vertices and may be followed either way.
- In a **directed graph**, an edge (u, v) is an ordered pair. It points from u to v , and (v, u) is a different edge.
- The word **simple** rules out self-loops and repeated parallel edges.
- An edge may carry a weight such as distance, capacity, time, or cost.

Modeling decision: direction records which transitions are feasible; weight records a quantity associated with an edge.

Walks, Paths, and Cycles

Let $v_0, v_1, \dots, v_k$ be a sequence of vertices.

- A **walk** follows an edge between every consecutive pair. A directed walk must follow every source-to-target direction.
- A **path** is a walk with no repeated vertices.
- A **cycle** is a closed walk with at least one edge that repeats no edge and no vertex except $v_0 = v_k$.

A path never revisits a vertex. A cycle returns to its starting vertex.

Connectivity Depends on Direction

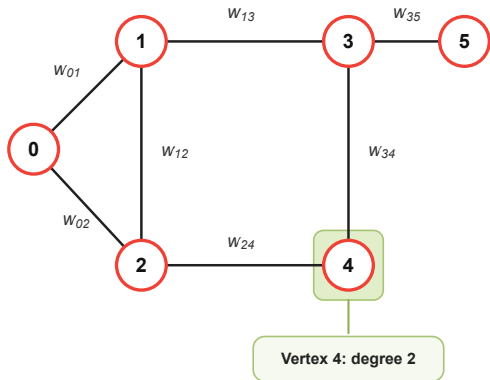
Connectivity asks whether paths join the vertices.

- An undirected graph is **connected** when a path joins every pair of vertices.
- A directed graph is **weakly connected** when ignoring edge directions produces a connected undirected graph.
- A directed graph is **strongly connected** when every ordered pair (u, v) is joined by a directed path from u to v .

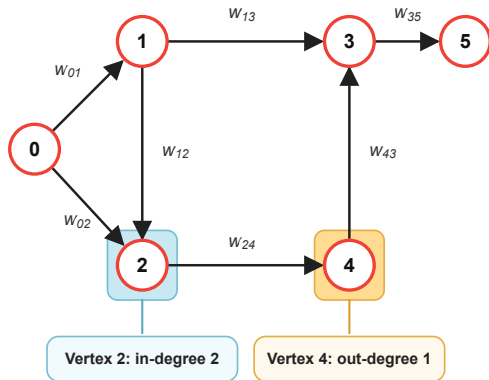
A **directed acyclic graph** has no directed cycles.

Direction Changes Which Routes Are Feasible

Undirected graph



Directed acyclic graph



Undirected edges allow travel either way; directed edges follow the arrows.
The directed graph is acyclic and weakly connected, but not strongly connected.

Directed Degree Has Two Parts

Count the edges touching vertex 4. Only its incident edges are shown.

Undirected



Two edges touch vertex 4.

$$\deg(v_4) = 2$$

Directed



One edge enters; one leaves.

$$\deg^{\text{in}}(v_4) = 1, \quad \deg^{\text{out}}(v_4) = 1$$

$$\text{Total degree: } \deg(v_4) = \deg^{\text{in}}(v_4) + \deg^{\text{out}}(v_4) = 2.$$

Degree Counts Recover the Edge Count

Let $n = |\mathcal{V}|$ and $m = |\mathcal{E}|$. In an undirected graph, summing degrees counts both endpoints of every edge:

$$\underbrace{\sum_{v_i \in \mathcal{V}} \deg(v_i)}_{\text{all incident ends}} = 2m.$$

In a directed graph, each edge contributes once to each directional sum:

$$\underbrace{\sum_{v_i \in \mathcal{V}} \deg^{\text{in}}(v_i)}_{\text{all targets}} = \underbrace{\sum_{v_i \in \mathcal{V}} \deg^{\text{out}}(v_i)}_{\text{all sources}} = m.$$

These **handshaking identities** are useful consistency checks for graph data.

Average Degree Lies Between the Extremes

For an undirected graph with $n \geq 1$, let $\delta(\mathcal{G})$ and $\Delta(\mathcal{G})$ be its minimum and maximum degrees. The average degree is given by:

$$\underbrace{\bar{d}(\mathcal{G}) = \frac{1}{n} \sum_{v_i \in \mathcal{V}} \deg(v_i)}_{\text{average degree}} = \frac{2m}{n}.$$

It is bounded by:

$$\delta(\mathcal{G}) \leq \bar{d}(\mathcal{G}) \leq \Delta(\mathcal{G}).$$

An undirected graph is **regular of degree r** when every vertex has degree r . Then $\delta = \bar{d} = \Delta = r$.

Density Measures the Fraction of Possible Edges

A simple loop-free graph on $n \geq 2$ vertices has maximum edge count:

$$|\mathcal{E}|_{\max} = \begin{cases} n(n-1)/2, & \text{undirected,} \\ n(n-1), & \text{directed.} \end{cases}$$

The density is the observed fraction of that maximum:

$$\underbrace{\rho(\mathcal{G}) = \frac{|\mathcal{E}|}{|\mathcal{E}|_{\max}}}_{\text{graph density}}, \quad 0 \leq \rho(\mathcal{G}) \leq 1.$$

Density near one is **dense**; density near zero is **sparse**. This lecture sets $\rho(\mathcal{G}) = 0$ when $n < 2$.

Beyond Vertex and Edge Counts

A path graph forms a single chain. A star graph connects every outer vertex to one central vertex. For $n \geq 4$, both have n vertices and $n - 1$ edges.

Their average degree and density agree, but distances between vertices differ. For a connected undirected graph, the **diameter** $\text{diam}(\mathcal{G})$ is the largest shortest-path distance, measured in edges.

We compare a path and a star on six vertices to see what edge counts alone cannot tell us.

Path and Star Graphs

Both graphs have $n = 6$ vertices and $n - 1 = 5$ edges.

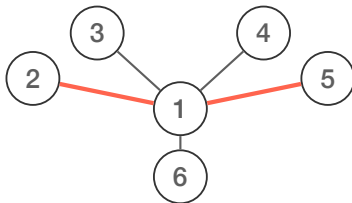
Path graph



Maximum degree: $\Delta = 2$

Diameter: $n - 1 = 5$ edges

Star graph



Maximum degree: $\Delta = n - 1 = 5$

Diameter: 2 edges

The **highlighted route** joins a most distant pair of vertices.

The graph is a mathematical object. Its representation determines what an algorithm can access efficiently.

Three Forms Store the Same Graph

Let $n = |\mathcal{V}|$ and $m = |\mathcal{E}|$.

- An **edge list** stores one source-target-weight record per edge.
- An **adjacency matrix** stores one entry for every ordered pair of vertices.
- An **adjacency list** stores one outgoing-neighbor collection per vertex.

Each form can describe the same graph, provided its vertex set is retained. The representation affects edge lookup and neighbor iteration.

Edge Lists Store One Record per Edge

An edge list stores each edge's endpoints and, when present, its weight.

- Edge records require $\Theta(m)$ storage. An explicit vertex set adds $\Theta(n)$, giving $\Theta(n + m)$ overall.
- Isolated vertices do not appear in edge records and must be stored separately.
- Without an index, finding an edge or collecting a vertex's neighbors takes $\Theta(1 + m)$ time in the worst case.

Edge lists suit reading, writing, and scanning the complete edge collection.

Adjacency Matrices Reserve Every Vertex Pair

For vertex order $v_1, \dots, v_n$, an unweighted adjacency matrix has entries:

$$a_{ij} = \begin{cases} 1, & \text{an edge joins } v_i \text{ to } v_j, \\ 0, & \text{otherwise.} \end{cases}$$

In a weighted matrix, an existing edge's entry stores its weight w_{ij} .

- Storage is $\Theta(n^2)$ and one edge lookup is $\Theta(1)$.
- Listing neighbors of v_i scans row i in $\Theta(n)$ time.
- An undirected matrix is symmetric; a directed matrix need not be.

Zero weights: zero can mark a missing edge only when it is not a valid edge weight; otherwise use a separate marker.

Adjacency Lists Store Neighbors

An adjacency list stores one collection of outgoing neighbors for each source.

- Unweighted form: `source => [target, ...]`.
- Weighted form: `source => [(target, weight), ...]`.
- Storage is $\Theta(n + m)$ for a directed graph.
- Accessing a source's dictionary entry is expected $\Theta(1)$; iterating over its out-neighbors costs $\Theta(1 + \deg^{\text{out}}(v_i))$.

In an undirected list, each edge appears in both endpoint lists. The exact count is $n + 2m$ vertex and neighbor records, which remains $\Theta(n + m)$.

Storage and Access Costs

Let $d_u = \deg^{\text{out}}(u)$. For a directed graph:

Representation	Storage	Find (u, v)	List neighbors
Edge list + vertex set	$\Theta(n + m)$	$\Theta(1 + m)$	$\Theta(1 + m)$
Adjacency matrix	$\Theta(n^2)$	$\Theta(1)$	$\Theta(n)$
Adjacency list	$\Theta(n + m)$	$\Theta(1 + d_u)$	$\Theta(1 + d_u)$

Edge searches use worst-case scan lengths; dictionary access is expected constant time. The added 1 includes the work needed for an empty list.

Scan assumption: the adjacency-list edge search scans the source vertex's neighbor vector.

Choose the Representation from the Operation

Does the edge (u, v) exist? Adjacency matrix

Read A_{uv} directly in $\Theta(1)$ time. Repeated edge lookups favor a matrix when its $\Theta(n^2)$ storage is affordable, especially for dense graphs.

Which vertices are neighbors of u ? Adjacency list

Scan the d_u neighbors of u instead of all n matrix entries. For a sparse graph, this reduces neighbor-search work and uses $\Theta(n + m)$ storage.

How do we process every edge? Edge list

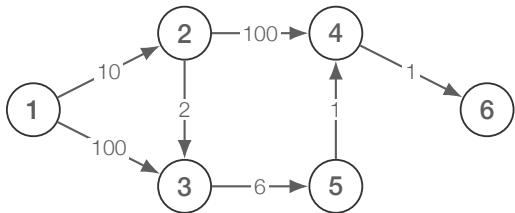
Read, write, or sort the edges as records. A full scan takes $\Theta(m)$ time, but finding one vertex's neighbors also requires scanning the list.

Balance the work done most often against the storage available.

Worked Example: A Six-Vertex Directed Graph

The file `SimpleGraph.txt` contains seven weighted edge records. The connections match the earlier directed graph; labels 0–5 become 1–6.

source	target	weight
1	2	10
1	3	100
2	3	2
2	4	100
3	5	6
4	6	1
5	4	1



Notebook functions: `read_weighted_edges`, `adjacency_list`, and `adjacency_matrix`.

The Same Edges Become a List or a Matrix

Outgoing adjacency list

1 $\rightarrow$ [2, 3]

2 $\rightarrow$ [3, 4]

3 $\rightarrow$ [5]

4 $\rightarrow$ [6]

5 $\rightarrow$ [4]

6 $\rightarrow$ []

Weighted adjacency matrix

	1	2	3	4	5	6
1	0	10	100	0	0	0
2	0	0	2	100	0	0
3	0	0	0	0	6	0
4	0	0	0	0	0	1
5	0	0	0	1	0	0
6	0	0	0	0	0	0

This lecture's helper stores targets only; the original edge records retain the weights.

Vertex Labels and Matrix Indices

Relabel the same six vertices as 10, 20, 30, 40, 50, 60; keep all edges and weights.
Choose `vertex_order = [10, 20, 30, 40, 50, 60]`.

Index-to-vertex mapping

Index i	Vertex label
1	10
2	20
3	30
4	40
5	50
6	60

Weighted adjacency matrix

	1	2	3	4	5	6
1	0	10	100	0	0	0
2	0	0	2	100	0	0
3	0	0	0	0	6	0
4	0	0	0	0	0	1
5	0	0	0	1	0	0
6	0	0	0	0	0	0

$A_{1,3} = 100$ records the edge $10 \rightarrow 30$ with weight 100.

Pair Lists with an Edge-Weight Map

A flat dictionary can map a known endpoint pair directly to a weight:

$$(u, v) \mapsto w(u, v).$$

- Expected lookup for a known (u, v) is $\Theta(1)$.
- Finding all neighbors of u still requires scanning m entries.
- Pairing the map with an unweighted adjacency list gives fast direct weight lookup and fast neighbor iteration.

Design principle: one graph object may maintain more than one representation when its algorithms require different access patterns.

Graph Storage: Counts and Growth

Our directed example has $n = 6$, $m = 7$, and $\rho = 7/[6(6 - 1)] = 7/30$.

Graph	Adjacency matrix	Adjacency list
Six-vertex example	$n^2 = 36$ entries	$n + m = 13$ items
Sparse: $m = 10n$	$\Theta(n^2)$	$\Theta(n + 10n) = \Theta(n)$
Dense: $m = \Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$

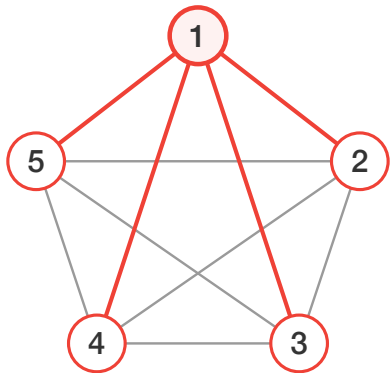
The matrix reserves space for absent edges. The example's list stores 6 vertex collections and 7 neighbor entries.

Big Theta describes storage growth as well as running time. With 10 outgoing edges per vertex, doubling n approximately doubles list storage but quadruples matrix storage.

Three graph families turn structural definitions into
recognition tests, edge counts, and storage requirements.

Complete Graphs Use Every Possible Edge

A complete graph K_n is a simple undirected graph with an edge between every pair of distinct vertices.



What the picture shows

Degree of every vertex

$$\deg(v) = 5 - 1 = 4$$

Number of edges

$$|\mathcal{E}| = \frac{5(4)}{2} = 10$$

Fraction of possible edges

$$\rho(K_5) = 1$$

— edges incident to vertex 1

Complete-Graph Measures Follow Directly

For $n \geq 2$, every vertex meets $n - 1$ others. Counting both endpoints of each edge gives:

$$|\mathcal{E}| = \binom{n}{2} = \frac{n(n-1)}{2}, \quad \deg(v_i) = n - 1.$$

Every possible edge is present, and each pair of distinct vertices is one edge apart:

$$\rho(K_n) = 1, \quad \text{diam}(K_n) = 1.$$

These counts describe connectivity and distance. Next, we consider groups of vertices and the colors needed to distinguish adjacent vertices.

Clique, Chromatic, and Independence Numbers

For an undirected graph:

- **Clique number** $\omega(\mathcal{G})$: the largest number of vertices that are all pairwise adjacent.
- **Chromatic number** $\chi(\mathcal{G})$: the fewest colors that place different colors on adjacent vertices.
- **Independence number** $\alpha(\mathcal{G})$: the largest number of vertices with no edge between any selected pair.

Every pair in K_n is adjacent, so for $n \geq 2$:

$$\omega(K_n) = \chi(K_n) = n, \quad \alpha(K_n) = 1.$$

Round-robin schedules have the edge pattern of K_n . Traveling-salesman models use a weighted K_n when every city pair has a symmetric travel cost.

Storage: since $|\mathcal{E}| = \Theta(n^2)$, both matrices and adjacency lists require $\Theta(n^2)$ storage.

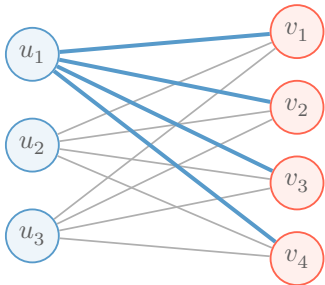
Bipartite and Complete Bipartite Graphs

A bipartite graph has edges only between two disjoint vertex groups. The complete bipartite graph $K_{m,n}$ includes every edge between them.

The graph $K_{3,4}$

$\mathcal{V}_1: m = 3$

$\mathcal{V}_2: n = 4$



Count each edge from the left

Each of m left vertices connects to all n right vertices:

$$|\mathcal{E}| = mn = 3(4) = 12.$$

Degree counts the opposite group

$$\deg(v) = \begin{cases} n = 4, & v \in \mathcal{V}_1, \\ m = 3, & v \in \mathcal{V}_2. \end{cases}$$

The four blue edges meet u_1 .

For $m, n \geq 1$, $K_{m,n}$ is regular exactly when $m = n$; $K_{3,4}$ is not regular.

Three Equivalent Tests Identify Bipartite Graphs

For an undirected graph $\mathcal{G}$, the following statements are equivalent:

1. $\mathcal{G}$ is bipartite.
2. $\mathcal{G}$ can be colored with at most two colors, so $\chi(\mathcal{G}) \leq 2$.
3. $\mathcal{G}$ contains no odd-length cycle.

Start a traversal in each uncolored component. Give each uncolored neighbor the opposite color and reject any same-color edge. With an adjacency list, the test takes worst-case $\Theta(|\mathcal{V}| + |\mathcal{E}|)$ time.

Bipartite Matrices Have Block Structure

Order the m vertices of $\mathcal{V}_1$ before the n vertices of $\mathcal{V}_2$. The $m \times n$ **biadjacency matrix** $\mathbf{B}$ records connections from $\mathcal{V}_1$ to $\mathcal{V}_2$. For an undirected graph:

$$\mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{B} \\ \mathbf{B}^\top & \mathbf{0} \end{bmatrix}.$$

In a directed graph, an $n \times m$ matrix $\mathbf{C}$ records the reverse connections, which can differ:

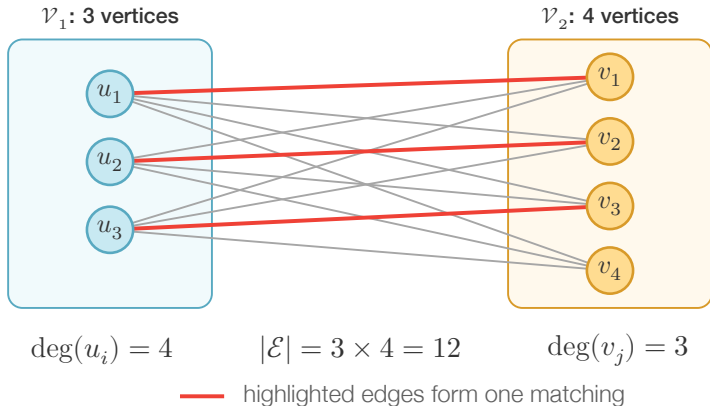
$$\mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{B} \\ \mathbf{C} & \mathbf{0} \end{bmatrix}.$$

The diagonal blocks are zero because no edge stays within a group. If every edge points from $\mathcal{V}_1$ to $\mathcal{V}_2$, then $\mathbf{C} = \mathbf{0}$.

Undirected storage: for $K_{3,4}$, $\mathbf{B}$ is a 3×4 matrix of ones; $\mathbf{B}$ and the vertex order determine $\mathbf{A}$.

A Matching Covers the Smaller Group

A **matching** is a set of edges that share no vertices. Here, the highlighted edges cover $\mathcal{V}_1$, leaving v_4 unmatched.



Matchings and Hall's Theorem

A matching that covers $\mathcal{V}_1$ assigns a different right-side option to every left-side vertex. Every group on the left must have enough neighbors to receive distinct assignments.

For $S \subseteq \mathcal{V}_1$, let $N(S) \subseteq \mathcal{V}_2$ be all neighbors of vertices in S . A matching covering $\mathcal{V}_1$ exists if and only if:

$$|N(S)| \geq |S| \quad \text{for every } S \subseteq \mathcal{V}_1.$$

Every group on the left must collectively have at least as many available options on the right. A **perfect matching** covers both groups and therefore requires $|\mathcal{V}_1| = |\mathcal{V}_2|$ together with Hall's condition.

Hall's theorem: [Mathematics for Computer Science, MIT](#)

Trees Are Minimally Connected Graphs

A tree $\mathcal{T} = (\mathcal{V}, \mathcal{E})$ is a connected undirected graph with no cycles.

- Exactly one path joins every pair of vertices.
- Removing any edge disconnects the tree.
- Adding an edge between nonadjacent vertices creates exactly one cycle.

A tree has exactly enough edges to remain connected and no edge that can be removed without losing connectivity.

Tree Edge Counts, Density, and Storage

Start with $n \geq 1$ isolated vertices. Each tree edge joins two components; joining vertices already connected would create a cycle. Reaching one component therefore requires $m = n - 1$ edges.

For $n \geq 2$, the tree's density is given by:

$$\rho(\mathcal{T}) = \frac{n - 1}{n(n - 1)/2} = \frac{2}{n}.$$

As $n \rightarrow \infty$, $\rho(\mathcal{T}) \rightarrow 0$: the tree stays connected while becoming sparse.

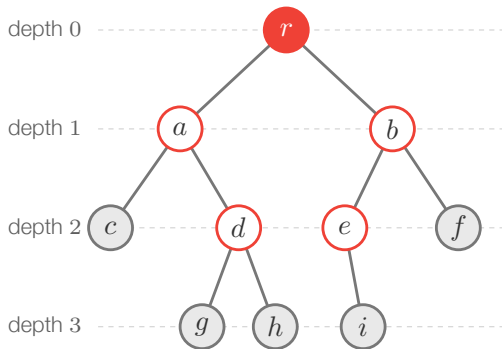
Adjacency-list storage is $\Theta(n)$ because $m = n - 1$.

Spanning Trees Retain Connectivity

Every connected undirected graph contains a **spanning tree**: a subgraph with all its vertices that remains connected and has no cycles.

- A spanning tree on n vertices contains exactly $n - 1$ edges.
- A **forest** is an undirected graph with no cycles.
- Each connected component of a forest is a tree.

A Root Gives a Tree Parent-Child Structure



Legend

- Root r**
has no parent; depth is 0
- Every other vertex**
has exactly one parent
- Leaves c, f, g, h, i**
have no children
- Height = 3**
the largest vertex depth

Every non-root vertex has one parent. Vertices may have children; those with no children are leaves. Depth counts edges from the root, and height is the maximum depth.

Reference: Equivalent Tree Characterizations

For a simple undirected graph with $n \geq 1$, these statements are equivalent:

1. The graph is a tree.
2. It is connected and has $n - 1$ edges.
3. It has no cycle and has $n - 1$ edges.
4. It is connected, and removing any edge disconnects it.
5. It has no cycle, and adding any missing edge creates exactly one cycle.
6. Every pair of vertices is joined by exactly one path.

Once connectivity is known, an edge count of $n - 1$ is enough to establish that the graph is a tree.

Summary

We described graph structure, examined three graph families, and compared representations for storing their connections.

Key takeaways

- **Graph structure and measures.** We used connectivity, degree, and density to describe graphs, then compared arrangements with the same vertex and edge counts.
- **Storage and access.** We compared edge lists, matrices, and adjacency lists to explain their memory use and access costs.
- **Graph families.** We related complete graphs, bipartite graphs, and trees to edge counts, coloring, matchings, and unique paths.

L4b turns an adjacency list into a systematic visit order; L4c uses weights and minimizes route cost.

That's a wrap. Which graph representation would you choose for the next algorithm, and what operation justifies that choice?